Hunter Worssam
September 11th, 2025

Evaluating the Effectiveness of Pre-trained Models from Different Hubs
Students should explore the comparative effectiveness of pre-trained models sourced from Kaggle, PyTorch Hub, TensorFlow Hub, and Hugging Face. Your discussion should consider critical factors including performance metrics (accuracy, precision, recall, and F1-score), computational resources required (memory usage, GPU/CPU demands, training time), and task suitability (e.g., NLP, computer vision, multimodal tasks). Additionally, discuss best practices in selecting appropriate pre-trained models, outline effective strategies for fine-tuning these models to specific applications, and address relevant ethical considerations and interpretability concerns that might influence model selection and deployment.

Background

Pre-trained models have become an invaluable resource when building machine learning workflows. These models, which are trained extensively on large-scale datasets, capture complex feature representations that can be transferred across domains and tasks. Leveraging pre-trained models significantly reduces computational demands, shortens training cycles, and enhances performance, particularly in data or compute constrained environments.

Several model hubs have emerged as central platforms to distribute, fine-tune, and deploy pre-trained models. Kaggle provides a competition-driven platform with a strong emphasis on reproducibility and community collaboration. In addition to publicly available datasets, it offers pre-trained models that enable rapid prototyping and benchmarking.
PyTorch Hub serves as a research-oriented repository, hosting widely used architectures such as ResNet, EfficientNet, and YOLO. Its integration with the PyTorch framework makes loading, reproducibility, and fine-tuning of state-of-the-art models a straightforward process.
TensorFlow Hub, developed by Google, prioritizes reusability and efficient transfer learning by storing models as reusable modules. With strong support for Keras integration and deployment pipelines, it is naturally well suited for computer vision, NLP, and mobile applications.
Hugging Face Hub has rapidly become the leading platform for natural language processing and multimodal learning. It offers thousands of transformer-based models including BERT, GPT, RoBERTa, and CLIP; along with tools for fine-tuning, hyper-parameter optimization, and deployment through APIs.

Choosing an appropriate pre-trained model depends on project-specific criteria. Key factors include the problem type (e.g., classification, regression, or generation), available computational resources, data size, and deployment environment (cloud or mobile for example). Smaller, resource-efficient models such as DistilBERT are suitable for constrained environments, whereas large transformer-based models can yield

superior accuracy when resources permit. In general, models with less than or equal to ~2-3 billion can be run on CPU, but models larger in size will require additional hardware.

Also important is the consideration of interpretability, reproducibility, and ethical constraints. Hugging Face promotes transparency through model cards, while best practices outlined across all of these platforms emphasize careful hyper-parameter tuning, validation, and systematic experiment tracking.

Collectively these ecosystems have accelerated the standardization of model building pipelines across the full landscape of model applications. At the end of the day, it will be up to the discretion of the user whether or not one of these pre-trained models will suffice for the problem at hand; but regardless, having access to these cutting edge architectures will inspire additional creativity during the model building process and facilitate a faster transition from experimentation to deployment.

Datasets Used

Two unique datasets were used for this analysis, with each being fed into two different models to compare the different model ecosystems.

Hugging Face IMDb NLP Dataset

- ~50,000 texts with either positive/negative sentiment)
- x = raw text, y = sentiment (positive or negative)

Kaggle CIFAR-10 Computer Vision Dataset

- 60,000 32x32 images with 10 unique classes
- x = Image, y = class label (e.g. airplane, automobile, bird, cat, …)

Models Used

For each of the two datasets above, two models were pulled the four main ecosystems (Kaggle, Hugging Face, PyTorch or Tensor flow), and the two models were compared at a baseline level, and after some light tuning. This produced pairwise comparisons per dataset in both baseline and tuned settings.

For the IMDb NLP Dataset

DistilBERT (Hugging Face Transformer)

- Compact, distilled variant of BERT pre-trained on English text.
- Setup: Uncased tokenizer, max_len=128.
- Baseline: Frozen backbone (train classifier only).
- Tuned: Unfreeze the last transformer layer + classifier head with small learning rates.

TF-IDF + Logistic Regression (Kaggle scikit-learn approach)

- Standard n-gram-based approach with no pre-training.
- Setup: TF-IDF with 1-2-grams feeding a logistic regression classifier.
- Baseline: 1-2 grams.
- Tuned: Grid search over max_features ∈ {20k, 40k, 80k} and ngram_range ∈ {(1,1),(1,2)} with Logistic Regression.

For the CIFAR-10 Computer Vision Dataset

ResNet-18 (PyTorch)

- A lightweight residual CNN pre-trained on ImageNet.
- Setup: Normalize to ImageNet mean/std; 256→224 geometry; no augmentation.
- Baseline: Replace fully connected head, freeze backbone
- Tuned: Unfreeze layer4 and run a small LR/weight-decay sweep.

MobileNetV2-1.3 (TensorFlow Hub)

- Mobile-optimized CNN with depth-wise separable convolutions.
- Setup: Scale pixels to [0,1]; 256→224 geometry; tf_keras API.
- Baseline: Frozen feature vector (train Dense(10) head) with LR ∈ {1e-3, 5e-4}.
- Tuned: Trainable backbone with tiny LR (≈1e-5)

Questions to Answer

P**erformance under pre & post tuning**

For each dataset (IMDb, CIFAR-10), which model performs best when evaluated as a baseline and after some light fine-tuning?
Winning model is selected by test F1 score. Model choice during tuning is made on validation F1 score; test score is reported once for the chosen config.

Efficiency (accuracy vs train time)

Across models per task, which model provides the best F1 score per training minute (F1_per_min). Rank by F1_per_min; and use this rate to analyze accuracy-vs-cost trade-offs.

Practicality & reproducibility across hubs

Given standardized preprocessing (no augmentation; CIFAR 256→224; DistilBERT max_len=128), what issues affected training and reproducibility across hubs (Hugging Face, PyTorch, TF-Hub, scikit-learn)?

Experimental Design

All of the following details can be found in the accompanying Jupyter notebook file.

A model comparison was performed on two benchmarks: IMDb (binary sentiment, text) and CIFAR-10 (10-class images); using one classical baseline and one pretrained

model per task. Each model was evaluated as a baseline and then with a light fine-tune under the same small training budget. All choices below were fixed before training to keep the comparison fair and reproducible.

Datasets & splits:
IMDb: used the standard train/test (25k/25k), dropped the “unsupervised” split, and held out 10% of train for validation.
CIFAR-10: used the standard train/test (50k/10k) with a 5k validation split carved from train; test remained untouched.

Preprocessing (standardized across models):
CIFAR-10 dataset (both ResNet-18 and MobileNetV2): Resize 256 → CenterCrop 224, no augmentation; ResNet normalized to ImageNet mean/std; MobileNetV2 inputs scaled to [0,1].

IMDb dataset: DistilBERT used uncased tokenizer, max_len=128, truncation + dynamic padding. NLP traditional approach (TF-IDF+LogReg): same base text cleanup as DistilBERT; 1–2-gram features.

Training protocols using a fixed budget:

- Baselines: replace/add a small classification head, freeze the backbone, train for 3 epochs.
- Light fine-tune: unfreeze a small portion with small learning rates for 3 epochs:
  - DistilBERT: unfreeze last transformer layer + head.
  - ResNet-18: unfreeze layer4 + head.
  - MobileNetV2-1.3: one run with tiny learning rate and smaller batch
  - TF-IDF+LogReg: small grid over max_features $\in$ {20k, 40k, 80k} and ngram_range $\in$ {(1,1),(1,2)}.

Model selection used validation F1; the test set was evaluated once for the selected configuration.

Tuning Hyper-parameters:
DistilBERT: head LR $\in$ {1e-4, 2e-4}, last-layer LR $\in$ {3e-5, 5e-5}, weight decay $\in$ {0, 0.01}.

ResNet-18: head LR $\in$ {1e-3, 3e-4}, layer4 LR $\in$ {1e-4, 5e-5}, weight decay $\in$ {0, 1e-4}.

MobileNetV2-1.3: frozen head LR $\in$ {1e-3, 5e-4}; trainable backbone LR ≈1e-5 with a smaller batch.

TF-IDF+LogReg: See training protocol section above.

Metrics & efficiency:
The primary mode evaluation metric used to assess model performance was F1 score, and efficiency was reported as F1 per training minute (same epoch budget for all runs).

This metric helps the user determine if additional training time is worth the added accuracy boost. Inference latency was not part of the original scope; if added later, it should be measured with a fixed device and batch for fairness.

Environment & reproducibility:
Runs were executed on Apple-silicon with Metal (TF) and MPS (PyTorch). Important import versions were recorded, seeds were set, and preprocessing steps were logged for each model. Additionally, identical geometry was used to ensure an apples-to-apples comparison within each dataset. The model-specific preprocessing steps did contain some differences to ensure the resulting models could ingest the data properly, however these nuances were required and should not impact our ability to compare models across the same dataset.

Results

IMDB Dataset, DistilBERT(Hugging Face) vs Standard Kaggle TF-IDF + Log. Regression Approach

Hub	Model	Task	Phase	Acc	F1	TrainTime_min	F1_per_min
Hugging Face	DistilBERT	IMDb	baseline	0.817	0.816	8.61	0.95
Hugging Face	DistilBERT	IMDb	fine-tune	0.867	0.867	28.51	0.03
Kaggle-style	TF-IDF + LogisticRegression	IMDb	baseline	0.896	0.896	0.01	89.6
Kaggle-style	TF-IDF + LogisticRegression	IMDb	fine-tune	0.898	0.898	0.12	7.48

CIFAR-10 Dataset, MobileNetV2 (Tensor flow) vs ResNet18 (PyTorch)

Hub	Model	Task	Phase	Acc	F1	TrainTime_min	F1_per_min
TF Hub	MobileNetV2	CIFAR-10	baseline	0.857	0.859	4.19	0.205
TF Hub	MobileNetV2	CIFAR-10	fine-tune	0.910	0.910	35.37	0.026
PyTorch Hub	ResNet18	CIFAR-10	baseline	0.767	0.766	4.65	0.165
PyTorch Hub	ResNet18	CIFAR-10	fine-tune	0.898	0.898	19.80	0.045

Discussion

Starting with the IMDb dataset & model comparison, one thing that was immediately clear was the advantage that the simple TF-IDF + logistic regression approach had in terms of setup simplicity, speed and interpretability. Compared to the DistilBERT model from Hugging Face, the logistic regression approach trained nearly 10 times faster when accounting for the time spent pre-processing the TF-IDF vectors. In addition to this added speed, it achieved a higher F1 score than the DistilBERT model on this binary sentiment classification task.

The DistilBERT model was relatively easy to implement as well, however when implementing the tuning procedure over the final layer, each training permutation took about ~5 to 6 minutes to train. With four unique learning rates tested, and a final run on the test set using the hyper parameters with the best validation score, this resulted in

over 25 minutes spent on optimization for a final F1 score lower than the simpler approach. It's important to note that this is a lighter version of the BERT model, so it's quite possible the BERT model would be able to achieve higher F1 scores with a thorough hyper parameter tuning process.

Regarding the MobileNetV2 and ResNet18 model architectures from Tensor flow and PyTorch respectively, both models were time and compute intensive to train, but yielded impressive F1 scores as a result. I had more trouble getting the Tensor Flow MobileNet up and running compared to PyTorch, however both models accommodated hyper-parameter tuning by unfreezing the output layers. The MobileNet took noticeably longer to train and achieved a slightly higher F1 score as a result, which may be worthwhile for certain applications.

I ran these models together in the same Jupyter notebook on my Apple M4 MacBook Pro and experience frequent kernel crashes when importing the PyTorch and Tensorflow libraries at the same time. The workaround was to step back and derive a fresh conda environment with all libraries pre-installed, which cleared up all of the import issues I was having. Overall, I think both hubs offer sufficient documentation on the model architectures, enabling easy implementation and some light hyper-parameter tuning.

Best Practices

The number one takeaway I have from this process is to ensure your conda environment is properly setup with all required libraries ahead of time, especially the libraries required for Tensor flow and PyTorch. When trying to import these libraries together within my Jupyter notebook, I frequently encountered kernel crashes that made running PyTorch and TensorFlow models extremely cumbersome. Once I created a standalone environment that included all of these required libraries (torch, torch vision, datasets, transformers, tensorflow, tensorflow_hub, etc.), everything progressed smoothly.

Another important detail when comparing pre-tained models is to standardize and thoroughly document your pre-processing steps, outputs and training times to ensure your final comparisons are as apples-to-apples as possible. Often times the model architectures dictate slight differences in the input vectors to be able to work properly, however there is still an opportunity to perform a baseline text or image cleanup before moving into these model-specific pre-processing steps. In a similar vein, when performing hyper-parameter tuning across the models, it's best to try and keep the search space equivalent across the models to not accidentally inflate/deflate training time metrics.

References

685.701 Model Building PDF (Available in “Model Building” section of Course Information)

https://huggingface.co/datasets/mteb/imdb

https://www.kaggle.com/c/cifar-10/

https://huggingface.co/docs/transformers/en/model_doc/distilbert

https://www.kaggle.com/code/shubhamptrivedi/sentiment-analysis-on-imdb-movie-reviews

https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html

https://pytorch.org/hub/pytorch_vision_mobilenet_v2/